Week 4: Conquering Continuous Translation – Implementing CTC Loss

1. Introduction and Objectives for Week 4

By the end of Week 3, the project had reached Stage 2 of the pipeline: the LSTM network was constructed, trained, and demonstrated to be able to classify isolated, pre-segmented ASL signs. But the final aim of this project is not to create a stand-alone gesture dictionary, but a live Continuous Sign Language Translation (CSLT) system.

Week 4 was mainly aimed at beginning Stage 3 of the architecture. This necessitated a fundamental change in the way the neural network learns, which was initially a basic classification model but was now a Sequence-to-Sequence (Seq2Seq) model that was able to learn to interpret flowing and unsegmented video streams. In a bid to accomplish this, I used Connectionist Temporal Classification (CTC) Loss.

2. The Core Challenge: Co-articulation and The Boundary Problem

With an independent collection of sign language sample videos, each video initiates precisely at the moment the sign commences and terminates precisely at the moment the sign concludes. Consistently, in the real world, human communication is ongoing.

Once one signs a sentence, he/she does not take his hands back to a neutral position between each word. Co-articulation is the physical movement, which is necessary to pass out of Sign A to Sign B. These transition frames appear as random noise to a typical neural network- or worse, they appear as entirely new, unwanted signs. Also, normal networks need coherent data: they must be aware of what video frame is associated with which word. In video that is continuous it is virtually impossible to manually mark the start and end frame of each individual word in a sentence.

3. Enter Connectionist Temporal Classification (CTC)

In a bid to address the boundary problem, I restructured the model to use CTC Loss. CTC was originally designed as a scoring function to enable a neural network to synchronize an input sequence (our video frames) with an output sequence (our translated words) when the timing and alignment of the two are not known.

CTC can achieve this mathematical trick by adding the mathematical concept of a Blank Token to the vocabulary of the model.

As the LSTM works on the continuous flow of spatial data, it delivers a probability distribution of each of the individual frames. When a user provides the sign of "HELLO" in 15 frames, the model will produce the sign of HELLO 15 times. As the user transitions their hands to the next sign, the model becomes unsure and outputs the "Blank Token" to absorb the transition noise. The CTC algorithm is the algorithm that collapses the output by eliminating the same consecutive predictions and eliminating the blank tokens.

For example, a raw frame-by-frame output of [HELLO, HELLO, HELLO, <blank>, <blank>, WORLD, WORLD] is elegantly collapsed into the sequence [HELLO, WORLD].

4. Modifying the LSTM Architecture

Implementing CTC required several significant modifications to the backend architecture built in Week 3 using TensorFlow/Keras:

- **Vocabulary Expansion:** I updated the output dense layer of the LSTM to equal Vocabulary_Size + 1. This additional class strictly represents the CTC "Blank Token."
- **Changing the Loss Function:** I stripped out the standard categorical_crossentropy loss function. Because our input (e.g., 90 frames of video) is longer than our output (e.g., 3 translated words), cross-entropy no longer works. I replaced it with a custom ctc_batch_cost function, which calculates the probability of all possible alignments and minimizes the error between the predicted sequence and the true sequence.
- **Decoding the Output:** I implemented a CTC Decoder (specifically a greedy search decoder) to take the raw probability matrix generated by the LSTM and collapse it into a readable string of text (ASL glosses).

5. Initial Testing on Unsegmented Data

Having updated the architecture, I started to feed the model longer, unsegmented sequences of spatial data. The CTC training process is particularly more computationally intensive, since it computes probabilities over the time axis, yet the findings were an enormous breakthrough.

The model was able to start generating sequences instead of predicting an individual word per video. It was the first time in the development of the project, that I could do three consecutive signs in front of the web-cam and the model was able to interpret and collapse the continuous movement in an ordered list of raw words. (e.g., [STORE, I, GO]).